	Std::sort	QuickSelect1	QuickSelect2	CountingSort	Unique Values
1K	0.059998 ms	0.033489 ms	0.044324 ms	0.293252 ms	787
100k	9.1221 ms	2.9132 ms	2.5086 ms	2.89992 ms	3588
10M	700.888 ms	230.990 ms	220.235 ms	200.798 ms	5335

Std::sort Big-O Analysis:

- Time complexity:
  Best case: O(n log n)
  Average Case : O(n log n)
  Worst Case: O(n^2)

Explanation:

- Fewer Unique Values, More Copies: If there are fewer unique values in the dataset and more copies of each value, std::sort may perform slightly better. This is because, in such cases, the sorting algorithm may encounter fewer unique comparisons, leading to potentially faster sorting times.
- Impact of Unique Values: The number of unique values doesn't significantly impact the time complexity of std::sort. However, it can affect the overall performance in terms of memory usage and cache efficiency. If there are many duplicate values, the algorithm might optimize for these cases, leading to faster execution.

Hypotheses:

- Performance with Few Unique Values: With fewer unique values and more copies, the sorting algorithm may be able to optimize its comparison process, potentially reducing the number of comparisons required.
- Performance with Many Unique Values: With many unique values, the sorting algorithm may incur more comparisons during the sorting process. However, since std::sort uses efficient algorithms like quicksort and heapsort, it still maintains good performance overall, albeit with slightly higher overhead.

CountingSort Big-O Analysis:

- Time complexity:
  Best Case: O(n+k)
  Average Case: O(n+k)
  Worst Case: O(n+k)

Explanation: In counting sort, n is the number of elements in the input array, and k is the range of the input (the maximum value minus the minimum value). Counting sort first counts the occurrences of each element in the input array, which takes O(n) time. Then, it iterates over the range of values (k) to create a sorted output array, which also takes O(k) time. Therefore, the overall time complexity is O(n + k).

Discussion:

- If there are fewer unique values and more copies of each value in the input array, counting sort may perform exceptionally well. This is because counting sort doesn't rely on comparisons between elements, so having more copies of the same value doesn't affect its performance negatively. Instead, it just increases the count for that value, which can be done efficiently.
- The number of unique values significantly affects the time complexity of counting sort. If the range of values (k) is much smaller than the number of elements (n), counting sort becomes very efficient, as it can complete in linear time. However, if the range of values is close to or larger than the number of elements, the time complexity approaches O(n^2), making counting sort less efficient compared to other sorting algorithms like quicksort or mergesort.

Hypotheses:

- Performance with Few Unique Values: With fewer unique values and more copies, counting sort will likely perform very well, completing in linear time. This is because it doesn't rely on comparisons between elements, and the counting process can be done efficiently.
- Performance with Many Unique Values: If there are many unique values in the input array, the time complexity of counting sort increases, especially if the range of values is close to the number of elements. In such cases, counting sort may not be as efficient as other sorting algorithms that have a time complexity of O(n log n).

QuickSelect1 Big-O Analysis:

- Time complexity:
  Best Case: O(n^2)
  Average Case: O(n)
  Worst Case: O(n^2)

Explanation: QuickSelect1 algorithm is based on the QuickSort algorithm. In the worst-case scenario, when the pivot selection is not optimal, QuickSelect1 may exhibit a time complexity of O(n^2), similar to QuickSort's worst-case time complexity. This occurs when the partition process consistently chooses poorly, such as when the smallest or largest element is always chosen as the pivot.

Discussion:

- Impact of Pivot Selection: The choice of pivot significantly affects the performance of QuickSelect1. If the pivot selection strategy doesn't effectively partition the array into two roughly equal halves, the algorithm's performance suffers. In the provided code, a random pivot selection strategy is used to mitigate this issue, but it doesn't guarantee optimal partitioning in every case.
- Fewer Unique Values, More Copies: QuickSelect1's time complexity is primarily influenced by the effectiveness of the partitioning process rather than the uniqueness of values. However, having fewer unique values may increase the likelihood of suboptimal pivot selections, potentially worsening the algorithm's performance.

Hypotheses:

- Performance with Random Pivot: When a random pivot selection strategy is used, QuickSelect1 is expected to perform reasonably well on average, with a time complexity closer to O(n log n) due to the efficient partitioning. However, in specific cases where the partitioning consistently produces unbalanced partitions, the time complexity may degrade to O(n^2).
- Impact of Data Distribution: If the input data has a skewed distribution or many duplicate values, the probability of encountering scenarios where the partitioning process performs poorly increases. This can lead to longer execution times, especially if the duplicates are at the beginning or end of the array, affecting the choice of pivot.

QuickSelect2 Big-O Analysis:

- Time complexity:
  Best Case: O((n+k)^2)
  Average Case: O(n)
  Worst Case: O(n^2):

Explanation:

- This occurs when the pivot selection consistently divides the array into roughly equal halves, leading to a balanced partitioning. However, even with a balanced partitioning, the insertion sort within small subarrays contributes to O((n+k)^2) complexity.
- Average Case: On average, quick select performs well because it tends to select pivots that roughly divide the array in half. This leads to linear time complexity, with occasional worst-case scenarios mitigated by the random pivot selection.
- Worst Case: The worst-case scenario happens when the pivot selection consistently leads to highly unbalanced partitions, such as when the smallest or largest element is repeatedly chosen as the pivot. This results in the maximum possible number of comparisons and swaps, leading to quadratic time complexity.

Discussion:

- The algorithm combines elements of quicksort and insertion sort to efficiently find quartiles in a dataset.
- Quick select is employed to partition the array and find quartile positions efficiently, while insertion sort is used on small subarrays.
- The choice of algorithm depends on the size of the subarray being sorted. Quick select is suitable for large subarrays, while insertion sort is efficient for small subarrays.

Hypotheses:

- The choice of pivot selection method could influence the algorithm's performance.
- Implementing a more sophisticated insertion sort variant might improve overall performance.
- Optimizing the algorithm for specific types of data distributions could lead to better average-case performance.

Additional Observation:

During the project, I observed the importance of considering both algorithmic efficiency and implementation details when analyzing sorting and selection algorithms. While Big-O notation provides a high-level comprehension of algorithmic performance, factors such as data distribution, pivot selection strategies, and the presence of duplicate values can significantly impact real-world execution times. Additionally, I noticed the value of thorough testing and benchmarking to validate algorithm implementations across various input sizes and distributions, helping to identify potential bottlenecks and optimize performance. Also I found that the choice of algorithm can depend heavily on the specific characteristics of the data being processed. For example, while quicksort-based algorithms like QuickSelect1 and QuickSelect2 offer efficient average-case performance, they may exhibit suboptimal behavior in certain edge cases. Understanding the trade-offs between different algorithms and their suitability for different datasets is crucial for designing robust and efficient sorting and selection solutions.